

Estructuras de programación

Objetivos

- Comprender la diferencia entre análisis léxico, sintaxis y semántica.
- Identificar los componentes léxicos de Java.
- Construir y evaluar expresiones matemáticas y lógicas.
- Utilizar métodos de la biblioteca estándar de Java.
- Reconocer los distintos tipos de sentencias.
- Entender el concepto de flujo de control.
- Desarrollar algoritmos iterativos.
- Realizar trazas de ejecución.

Contenido

Introducción	3
Análisis léxico y componentes del lenguaje	3
Concepto de Token y separador	4
Palabras reservadas	13
Expresiones y Operadores	15
Operadores	15
Evaluación en cortocircuito	15
Precedencia y asociatividad	15
Conversión de tipos	15
Reglas de promoción	15
Conversión explícita	15
Clases envolventes	15
Uso de la biblioteca estándar	15
La clase Math	15
Instrucciones	15
Sentencias	15
Bloques y ámbito	15
El flujo de ejecución	15
Concepto de flujo de control	15
Estructuras secuenciales	15
Selección	15
Selección simple	15
Selección compuesta	15
Selección múltiple	15
Lógica de anidamiento	15
Repetición	15
Sentencia while	16
Bucles controlados por condición	16
Bucles controlados por contador	16
Diseño de bucles	16
Depuración y verificación	16
Trazas de ejecución	16
Errores comunes	16

Introducción

Hasta este punto de nuestro aprendizaje, hemos comprendido que un programa de ordenador es, en esencia, un algoritmo escrito para ser ejecutado por una máquina. En el tema anterior, exploramos los cimientos: aprendimos a declarar variables, a reconocer tipos de datos primitivos y a utilizar la biblioteca estándar de Java para realizar operaciones de entrada/salida básicas. Sin embargo, si analizamos los programas que hemos construido, observaremos que todos comparten una característica común: son puramente lineales. El ordenador se limita a ejecutar una instrucción tras otra, en el estricto orden físico en que fueron escritas.

No obstante, la programación en el «mundo real» y la resolución de problemas técnicos complejos en entornos productivos requieren una sofisticación mucho mayor. El pensamiento computacional no es una simple sucesión de pasos, sino un tejido o «urdimbre» donde cada hebra lógica debe entrelazarse con precisión. Para lograr esto, un lenguaje de programación debe ser entendido como un conjunto de reglas, símbolos y palabras que nos permiten modelar la realidad.

El proceso por el cual el ordenador interpreta nuestras intenciones comienza con el análisis léxico. Cuando el compilador recibe nuestro código fuente, su primera tarea es identificar elementos con significado propio. Como sucede en el lenguaje natural, no podemos formar una oración coherente amontonando palabras: en los lenguajes de programación combinamos estos componentes léxicos para formar expresiones. Algo como $n + 1$, por sí solo, es solo un valor latente; para que sea algo capaz de realizar una tarea, debe integrarse en una estructura que le dé sentido.

La verdadera potencia del software reside en la capacidad de romper la linealidad del código para establecer un flujo de control. Este define el orden en que las sentencias se ejecutan realmente durante la actividad de la aplicación. Para ilustrarlo, podemos usar la analogía de los músicos: ir tocando las notas según indica el director equivale a una estructura secuencial; sin embargo, podremos tomar una decisión basada en una condición o, quizás, que una serie de acordes se repitan.

Profundizaremos aquí en cómo construir expresiones y conoceremos la selección y la iteración, herramientas que nos permitirán empezar a desarrollar algoritmos inteligentes, capaces de tomar decisiones autónomas y procesar información de forma eficiente, sentando así las bases necesarias antes de adentrarnos en los fundamentos de la programación orientada a objetos.

Análisis léxico y componentes del lenguaje

Como hemos señalado, la construcción de un programa comienza con la escritura de **código fuente**, una tarea que realizamos utilizando un editor y siguiendo las reglas gramaticales del lenguaje. Sin embargo, para que este texto cobre vida y se convierta en una serie de acciones ejecutables, debe pasar por un proceso de transformación riguroso. El primer paso de este viaje técnico es el **análisis léxico**, una fase en la que el

compilador actúa como un lector meticuloso que descompone la secuencia ininterrumpida de caracteres que le entregamos en piezas individuales dotadas de significado.

En el lenguaje natural, cuando leemos una frase, nuestro cerebro identifica automáticamente dónde termina una palabra y comienza la siguiente, reconociendo sustantivos, verbos y adjetivos. En el ámbito de la programación, el ordenador realiza una operación análoga. No percibe el código como un conjunto de instrucciones complejas de un solo vistazo, sino como un flujo de símbolos pertenecientes al alfabeto; en el caso de Java, *Unicode*. La labor del analizador léxico consiste en agrupar estos símbolos en unidades atómicas, que son los componentes básicos que conforman la estructura del lenguaje.

Esta etapa es fundamental porque establece los cimientos sobre los que se construirá la lógica del programa, permitiendo ensamblar estos bloques en estructuras sintácticamente correctas. Si el analizador léxico encuentra un símbolo inesperado o una combinación de caracteres que no encaja en las categorías permitidas, el proceso se detendrá antes incluso de intentar comprender la intención del algoritmo. Por lo tanto, antes de profundizar en cómo tomar decisiones o repetir procesos, es esencial comprender la naturaleza de estos elementos mínimos y los mecanismos que permiten al compilador distinguirlos unos de otros.

Analizador léxico: Fase del compilador que descompone el flujo continuo de caracteres del código fuente en unidades atómicas con significado (*tokens*).

Concepto de Token y separador

Dentro del proceso de análisis léxico, el compilador debe ser capaz de reconocer los componentes básicos que integran el código fuente para poder interpretarlos adecuadamente. Estos componentes mínimos, que no pueden ser descompuestos en partes más pequeñas sin perder su sentido, reciben el nombre de ***tokens*** o elementos atómicos. En la gramática de un lenguaje de programación como Java, los *tokens* son el equivalente a las palabras en una oración del lenguaje natural; cada uno de ellos cumple una función específica, ya sea representar un valor constante, nombrar una variable o indicar una operación matemática.

Para que el analizador léxico pueda distinguir estos elementos dentro de la secuencia ininterrumpida de caracteres que escribimos, el lenguaje emplea unos símbolos especiales denominados **separadores**. Un separador es cualquier carácter o marca que indica de manera inequívoca dónde termina un *token* con significado propio y dónde comienza el siguiente. En este sentido, la programación vuelve a imitar a la lengua: así como usamos espacios para separar palabras en un texto escrito para que no se conviertan en una masa de letras incomprensible, Java utiliza los separadores para evitar que los *tokens* se fusionen.

Token: Unidad mínima o componente atómico que posee significado propio dentro de la gramática del lenguaje de programación.

Separador: Carácter o marca que delimita inequívocamente el fin de un *token* y el inicio del siguiente.

Los elementos que actúan como separadores en Java son los siguientes:

- **Espacio en blanco:** es el separador más común y sencillo, utilizado de la misma forma que en la escritura convencional.
- **Tabulador:** actúa de forma equivalente al espacio en blanco desde el punto de vista del compilador.
- **Retorno de carro o salto de línea:** indica el fin de una línea física, pero léxicamente tiene el mismo valor que un espacio. El compilador es capaz de tratar un programa completo escrito en una sola línea o con cada palabra en una línea distinta con el mismo resultado técnico, ya que sustituye cualquier secuencia de estos separadores por un único espacio en blanco¹.

Una vez que el analizador léxico ha hecho uso de estos separadores para aislar los *tokens*, estos se clasifican en diferentes categorías según su naturaleza, como los literales, los identificadores, los operadores y las palabras reservadas. Solo si el ensamblaje de estos bloques atómicos sigue las reglas de la sintaxis, el compilador podrá avanzar hacia la fase de traducción y ejecución del programa.

Literales

Un **literal** —o valor literal—, como ya vimos en el capítulo anterior, es la representación explícita y constante de un valor concreto directamente escrito en el código fuente de un programa. A través de los literales, el programador especifica valores fijos para los tipos de datos primitivos, el tipo de datos `String` y la referencia especial **`null`**.

Desde el punto de vista del análisis léxico, los literales son identificados por el compilador como *tokens* atómicos. Esto significa que constituyen unidades de información indivisibles con significado propio dentro de la gramática del lenguaje. A diferencia de lo que ocurre con un identificador o una variable, cuyo contenido o estado puede fluctuar dinámicamente durante el ciclo de vida de la ejecución, el valor representado por un literal queda prefijado de manera inmutable desde la fase de compilación.

Literal: *Token* que representa la expresión inalterable de un valor constante de un tipo de datos primitivo, de un objeto `String` o de la referencia nula **`null`** dentro del código fuente.

1. Como vimos en el capítulo anterior, esto no es exactamente así cuando se trata de una cadena, por ejemplo.

Dentro de la especificación sintáctica de Java, los literales se clasifican rigurosamente según la categoría de datos a la que corresponden, obedeciendo a un conjunto estricto de convenciones léxicas para su escritura. Plantilla sintáctica de declaración de literal:

```
Literal:  
  IntegerLiteral  
  FloatingPointLiteral  
  BooleanLiteral  
  CharacterLiteral  
  StringLiteral  
  NullLiteral
```

Literales enteros

De forma predeterminada, cuando el analizador léxico procesa un literal numérico compuesto únicamente por dígitos (sin coma ni punto fraccionario), el compilador deduce que se trata de un valor de tipo **int** codificado en 32 bits. La representación convencional se realiza en decimal, mediante una secuencia de dígitos del 0 al 9. Un literal decimal no debe comenzar con el dígito 0 (salvo el propio número cero).

```
IntegerLiteral:  
  DecimalNumeral  
  
DecimalNumeral:  
  0  
  NonZeroDigit [Digits]  
  
NonZeroDigit:  
  (one of)  
  1 2 3 4 5 6 7 8 9  
  
Digits:  
  Digit  
  Digit Digits  
  
Digit:  
  0  
  NonZeroDigit
```

Esta sucesión de reglas merece una pequeña explicación. Un literal de tipo entero, *IntegerLiteral*, es bien el número cero o un dígito distinto de cero, *NonZeroDigit*, seguido, opcionalmente, de dígitos, *Digits*. Estos dígito, a su vez, pueden ser un dígito *Digit* o un dígito seguido de dígitos. Por último, un dígito es el número cero o un número distinto de cero. La especificación lo que nos dice es que un número entero puede tener un número indeterminado de dígitos pero que no puede comenzar por cero, salvo que sea el cero.

A partir de Java 7, se permite intercalar el carácter de subrayado () entre los dígitos de cualquier literal numérico para mejorar la legibilidad del código fuente (por ejemplo, 1_000_000). Estos caracteres son ignorados por el analizador léxico durante la fase de compilación y no afectan al valor asignado.

```
Digits:
    Digit
    Digit [DigitsAndUnderscores] Digit

DigitsAndUnderscores:
    DigitOrUnderscore {DigitOrUnderscore}

DigitOrUnderscore:
    Digit
    —
```

Además del formato convencional, en decimal, Java permite representar literales enteros mediante otros tres sistemas de numeración:

- **Hexadecimal (base 16):** se denota mediante el prefijo 0x o 0X, aceptando dígitos del 0 al 9 y letras de la a a la f (tanto en mayúsculas como en minúsculas). Un ejemplo es el número 26: 0X1A.
- **Octal (base 8):** se especifica anteponiendo un cero (0) como prefijo al número, seguido exclusivamente por dígitos comprendidos entre el 0 y el 7 (por ejemplo, 012, que es el número 10).
- **Binario (base 2):** permite la notación explícita mediante el prefijo 0b o 0B, empleando únicamente los dígitos 0 y 1 (por ejemplo, el número 10 se representa como 0b1010²).

```
IntegerLiteral:
    DecimalNumeral
    HexNumeral
    OctalNumeral
    BinaryNumeral

HexNumeral:
    0 x HexDigits
    0 X HexDigits

HexDigits:
    HexDigit
    HexDigit [HexDigitsAndUnderscores] HexDigit

HexDigit:
    (one of)
    0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F
```

2. La capacidad de escribir literales en notación binaria directa mediante el prefijo 0b o 0B fue introducida formalmente en la versión Java SE 7 mediante la propuesta JEP 100.

```

HexDigitsAndUnderscores:
    HexDigitOrUnderscore {HexDigitOrUnderscore}

HexDigitOrUnderscore:
    HexDigit
    —

OctalNumeral:
    0 OctalDigits
    0 OctalDigits

OctalDigits:
    OctalDigit
    OctalDigit [OctalDigitsAndUnderscores] OctalDigit

OctaDigit:
    (one of)
    0 1 2 3 4 5 6 7

OctalDigitsAndUnderscores:
    OctalDigitOrUnderscore {OctalDigitOrUnderscore}

OctalDigitOrUnderscore:
    OctalDigit
    —

BinaryNumeral:
    0 b BinaryDigits
    0 B BinaryDigits

BinaryDigits:
    BinaryDigit
    BinaryDigit [BinaryDigitsAndUnderscores] BinaryDigit

BinaryDigit:
    (one of)
    0 1

BinaryDigitsAndUnderscores:
    BinaryDigitOrUnderscore {BinaryDigitOrUnderscore}

BinaryDigitOrUnderscore:
    BinaryDigit
    —

```

Si se requiere que el literal sea interpretado por el compilador como un entero de precisión extendida de 64 bits (**long**), es obligatorio añadir el sufijo **L** o **l** al final de la secuencia de dígitos³.


```
IntegerLiteral:  
  DecimalNumeral [IntegerTypeSuffix]  
  HexNumeral [IntegerTypeSuffix]  
  OctalNumeral [IntegerTypeSuffix]  
  BinaryNumeral [IntegerTypeSuffix]
```

```
IntegerTypeSuffix:  
  (one of)  
  l L
```

Internamente, la máquina virtual Java (JVM) almacena todos los valores enteros utilizando la representación binaria en **complemento a dos con signo**⁴. Este formato fija los límites inferior y superior de los rangos numéricos que cada tipo de entero puede albergar en memoria (por ejemplo, entre -2^{31} y $2^{31} - 1$ para el tipo **int**).

```
int contador = 100;           // Decimal  
int mascaraHex = 0xFF;       // Hexadecimal (255)  
int patronBits = 0b1100;     // Binario (12)  
long poblacion = 8000000000L; // Literal de 64b
```

Literales de punto flotante

Los literales de punto flotante se emplean para expresar números reales. Por defecto, la gramática de Java interpreta cualquier literal con punto decimal o notación exponencial como un valor de tipo **double** (64 bits), garantizando un grado elevado de precisión matemática.

Existen dos formas estándar de escribir literales de punto flotante:

- **Notación decimal:** utiliza el carácter punto (.) para separar explícitamente la parte entera de la fraccionaria (por ejemplo, 3.14159 o 5).
- **Notación científica o exponencial:** añade la letra e o E seguida de un número entero (positivo o negativo) que representa un exponente o potencia de diez (por ejemplo, 1.5e3 para representar 1.5×10^3 , es decir, 1500.0, o 2.5E-4 para 2.5×10^{-4}).

```
FloatingPointLiteral:  
  DecimalFloatingPointLiteral [FloatTypeSuffix]
```

3. Se recomienda enfáticamente el uso de la letra **L** mayúscula para evitar confusiones visuales con el número uno (1).

4. En la representación en complemento a dos, el bit más significativo, el primero, es 1 cuando el número es negativo. Los números negativos se calculan cambiando los unos por ceros y los ceros por unos, sumando uno al resultante.

```

DecimalFloatingPointLiteral:
  Digits . [Digits] [ExponentPart]
  . Digits [ExponentPart]
  Digits ExponentPart
  Digits [ExponentPart]

ExponentPart:
  ExponentIndicator SignedInteger

ExponentIndicator:
  (one of)
  e E

SignedInteger:
  [Sign] Digits

Sign:
  (one of)
  + -

FloatTypeSuffix:
  (one of)
  f F e E

```

A partir de las reglas anteriores podemos ver que un número en punto flotante puede empezar por punto cuando la parte entera sea cero. Cuando sea preciso declarar un literal de precisión simple de 32 bits (**float**) —para cumplir con los requerimientos de un método, por ejemplo—, se debe añadir el sufijo **f** o **F** al final del número.

```

double pi = 3.1415926535; // Tipo double por defecto
double masa = 5.972e24; // Notación científica (double)
float gravedad = 9.81f; // Sufijo f para forzar el tipo float

```

Supongamos que nuestra máquina, en la que cada ubicación de memoria tiene el mismo tamaño, permite representar los números mediante seis dígitos y un signo. Cuando definimos una variable o constante, la ubicación asignada consta de seis dígitos y un signo; si el valor es entero, la interpretación del número almacenado en esa ubicación es sencilla: puedo almacenar valores desde el -999999 hasta el +999999; el cero tiene dos representaciones. Nuestra precisión es seis dígitos y los números en ese rango pueden ser representados de forma exacta.

Supongamos que los dos dígitos de más a la izquierda nos permiten representar un exponente. La representación +561234, por ejemplo, es en realidad el número 1234×10^{56} . De hecho, el rango de números que ahora podemos representar es mucho mayor: desde -9999×10^{99} hasta $+9999 \times 10^{99}$. Sin embargo, la precisión ahora es de solo cuatro dígitos; es decir, solo los números de cuatro dígitos pueden representarse con exactitud en nuestro sistema. ¿Qué sucede con los números con más dígitos? Los cuatro dígitos de la izquierda se representan correctamente, y los dígitos de la derecha, o dígitos menos significativos, se pierden (se asume que son 0). Por ejemplo, 1000000

puede representarse con exactitud, pero 4932416 no, porque nuestro esquema de codificación nos limita a cuatro dígitos significativos.

Para extender nuestro esquema de codificación y representar números de punto flotante, debemos poder representar exponentes negativos. Dado que nuestro esquema no incluye un signo para el exponente, vamos a modificarlo ligeramente: el signo existente se convierte en el signo del exponente y añadimos un signo a la izquierda para representar el signo del número.

Ahora podemos representar con precisión, con cuatro dígitos, todos los números entre -9999×10^{99} y $+9999 \times 10^{99}$. Añadir exponentes negativos a nuestro esquema nos permite representar fracciones tan pequeñas como 1×10^{-99} . Nuestra precisión sigue siendo de cuatro dígitos. Los números 0.1032, 5.406 y 1000000 se pueden representar con exactitud. El número 476.0321, sin embargo, tiene siete cifras significativas, pero se representa como 476.0; ese 0.0321 no se puede representar en nuestro sistema⁵.

La codificación interna sigue la norma **IEEE 754** (formato binario de punto flotante para ordenadores), estructurando el valor en tres campos binarios diferenciados: bit de signo, mantisa y exponente. En el caso de los valores de tipo **float**, el bit de signo es +1 o -1, la mantisa es un número entero positivo menor que 2^{24} y el exponente en un número entre -126 y 127, incluidos. En el caso de los valores de tipo **double**, la mantisa es un número entero positivo menor que 2^{53} y el exponente en un número entre -1022 y 1023, incluidos.

El nombre *punto flotante* hace referencia a que el *punto decimal* puede moverse. En nuestro modelo, cada número se almacena en cuatro dígitos; si el dígito que no es cero se almacena en la posición de más a la izquierda, el exponente se ajusta de la manera correspondiente: por ejemplo, 1000000 podríamos almacenarlo como +060001 pero lo almacenamos como +031000. Este formato proporciona la máxima precisión posible, denominándose formato normalizado. En el modelo que usa la JVM, además, solo puede haber un único dígito distinto de cero a la izquierda del punto decimal, lo que nos permite aumentar el número de dígitos de la mantisa en uno, por lo que la precisión no es de 24 o 53 bits: pasa a ser 25 y 54. Esto es debido a que sabemos con certeza que el primer dígito siempre es 1 -al representarlo en base 2- y la máquina no lo almacena.

Es evidente que podemos representar valores extremadamente amplios o infinitesimales, asumiendo las limitaciones de redondeo y precisión propias de la aritmética de punto flotante. Sin embargo, con la explicación anterior, podríamos pensar que el cero no puede representarse. La realidad es que existen patrones para designar números específicos.

Y aunque pueda parecer otra cosa, hay que tener en cuenta que el número máximo de valores diferentes que la notación en punto flotante puede representar es 2^{32} o 2^{64} números: lo que hemos hecho es expandir esos números en un rango mayor. Además, los números que podemos representar no están espaciados de la misma manera entre ellos,

5. La JVM realiza redondeo en lugar de truncamiento al descartar los dígitos sobrantes. Si asumimos que tiene cuatro cifras significativas, el redondeo almacenaría 476.0321 como 476.0, pero almacenaría 476.0823 como 476.1.

como los números en punto fijo. Los posibles valores están más próximos cerca del origen y más separados cuanto más alejados.

Otros literales

Para la representación explícita de valores no numéricos o de estructuras complejas, el lenguaje provee la siguiente gama de literales:

- **Literales booleanos**
- **Literales de carácter**
- **Literales de cadena**
- **Literal nulo:** formado por la palabra reservada **null**, representa la ausencia explícita de referencia a un objeto dentro de una variable de tipo objeto o referencia.

```
BooleanLiteral:
  (one of)
  true false

CharacterLiteral:
  ' SingleCharacter '
  ' EscapeSequence '

SingleCharacter:
  JavaLetterOrDigit but not ' or </span>

EscapeSequence:
  t (horizontal tab HT, Unicode 009)
  n (linefeed LF, Unicode 00a)
  r (carriage return CR, Unicode 00d)
  </span> LineTerminator (line continuation, no Unicode representation)
  " (double quote", Unicode 022)
  ' (single quote', Unicode 027)
  </span> (backslash , Unicode 05c)

LineTerminator:
  the ASCII LF character, also known as "newline"
  the ASCII CR character, also known as "return"
  the ASCII CR character followed by the ASCII LF character

StringLiteral:
  " {StringCharacter} "

StringCharacter:
  JavaLetterOrDigit but not ' or </span> EscapeSequence

NullLiteral:
  null
```

```
boolean activo = true;
char inicial = 'J';
```

```
char salto = '\n';  
char unicodeA = '\u0041';  
String saludo = "Bienvenido a Java";  
Object objetoVacio = null;
```

Palabras reservadas

Como ya comentamos en el capítulo anterior, son aquellos *tokens* que tienen asignada una función específica dentro del lenguaje: los programadores tienen prohibido utilizarlas para cualquier otro propósito; por ello, no es posible nombrar una variable, un campo o una clase utilizando uno de estos términos. Al igual que en el lenguaje natural existen palabras con funciones gramaticales fijas, como las preposiciones, las palabras reservadas constituyen el vocabulario fundamental que el compilador reconoce para estructurar la lógica del programa.

Gran parte del proceso de aprendizaje de un lenguaje de programación consiste en familiarizarse con el significado y la utilidad de estos términos. En el caso de Java, el conjunto de estas palabras ha ido evolucionando con las diferentes versiones del lenguaje para adaptarse a nuevas necesidades técnicas. En la actualidad, el núcleo está compuesto por cincuenta y una palabras, a las que se suman dieciséis términos adicionales cuyo significado especial solo se activa dependiendo del contexto específico en el que se utilicen.

Es importante destacar que Java es un lenguaje extremadamente estricto con la forma en que se escriben estos elementos, siendo sensible a la diferencia entre mayúsculas y minúsculas. Todas las palabras reservadas se componen exclusivamente de letras minúsculas. Por esta razón, un término como **int** es reconocido como una palabra reservada para definir tipos enteros, mientras que **Int** o **INT** serían tratados como identificadores diferentes definidos por el usuario, aunque su uso se desaconseja para evitar ambigüedades en la lectura del código.

Dentro de este grupo, existen casos particulares como las palabras **goto** y **const**. Aunque figuran en la lista oficial y el programador no puede emplearlas como nombres de variables, actualmente no tienen ninguna función operativa dentro del lenguaje Java. Su reserva responde principalmente a razones históricas y al deseo de los diseñadores del lenguaje de evitar que programadores provenientes de otros entornos, como C o C++, intenten aplicar estructuras de programación que son incompatibles con la seguridad y la filosofía de Java.

A continuación, se presenta la relación de palabras reservadas en Java, estructurada conforme a la [Especificación del Lenguaje Java](#)⁶:

6. La documentación y especificación oficial actualizada de Java puede consultarse en la plataforma [Oracle Java Specification](#).

Palabras reservadas (*Keywords*) en la <https://docs.oracle.com/javase>

abstract	assert	boolean	break	byte	
case	catch	char	class	const	
continue	default	do	double	else	
enum	extends	final	finally	float	
for	goto	if	implements	import	
instanceof	int	interface	long	native	
new	package	private	protected	public	
return	short	static	strictfp ⁷	super	
switch	synchronized	this	throw	throws	
transient	try	void	volatile	while	
—					

Asimismo, existen los siguientes términos que funcionan como palabras reservadas dependiendo del contexto en el que se utilicen:

Palabras reservadas dependientes del contexto (*Contextual Keywords*)

exports	module	non-sealed	open	opens
permits	provides	record	requires	sealed
to	transitive	uses	var	when
with	yield			

Finalmente, conviene recordar que el carácter de subrayado () por sí solo también se considera un elemento reservado para usos futuros⁸, lo que prohíbe su empleo como un identificador de una sola letra en las declaraciones del programa.

7. La palabra strictfp fue introducida para restringir los cálculos de punto flotante a la norma IEEE 754; su uso es obsoleto.

8. A partir de Java 9 (JEP 213), el carácter de subrayado dejó de ser un identificador válido y pasó a ser un término reservado.

Expresiones y Operadores

Operadores

Evaluación en cortocircuito

Precedencia y asociatividad

Conversión de tipos

Reglas de promoción

Conversión explícita

Clases envolventes

Uso de la biblioteca estándar

La clase Math

Instrucciones

Sentencias

Bloques y ámbito

El flujo de ejecución

Concepto de flujo de control

Estructuras secuenciales

Selección

Selección simple

Selección compuesta

Selección múltiple

Lógica de anidamiento

Repetición

Sentencia while

Bucles controlados por condición

Bucles controlados por contador

Diseño de bucles

Depuración y verificación

Trazas de ejecución

Errores comunes